TIKO — Database Design

Oct 10, 2026 · @ki

1. Design decisions and conventions

TIKO uses one Postgres schema (Neon or Supabase) whose synced tables mirror the IndexedDB stores on each device, so the same row shape, ids and rules work in both places. It follows the Technical Specification; where that document and this one differ, fix the one that is wrong.

Decision	Rule	Why
Primary keys	`uuid`, generated on the client (UUID v7)	Offline creation, idempotent retries, guest merge with no id clashes
Tenancy	Every user-owned table has `user_id` with `ON DELETE CASCADE`	One delete removes an account's data (FR-SET-03)
Money	`bigint` in paise; no floats or decimals	Exact sums (FR-MON-03); currency lives on the user
Dates	`local_date date` in the user's time zone beside a UTC `timestamptz`	A time-zone change never rewrites history; day boundary is local midnight
Soft delete	`deleted_at timestamptz` tombstone on synced tables	Deletes must reach other devices
Conflicts	`updated_at` set by the client, clamped by the server; latest wins per row	FR-SYN-04
Change feed	`seq bigint` from one sequence, reassigned by trigger on every insert or update	Cursor-based pull; no clock dependence
Derived data	`goal.progress` is a cache; logs are the truth	Undo, edits and offline merges stay correct
Enums	Postgres enum types for closed sets	Cheap, validated, readable

Decision	Rule	Why
Sensitive text	Journal body stored only as ciphertext	NFR-06

Common columns on synced tables. `id uuid primary key`, `user_id uuid not null`, `created_at timestamptz not null default now()`, `updated_at timestamptz not null`, `deleted_at timestamptz`, `seq bigint not null`. They are not repeated in each table below, except where a table is not synced.

```
CREATE EXTENSION IF NOT EXISTS citext;

CREATE SEQUENCE change_seq;

CREATE FUNCTION bump_seq() RETURNS trigger AS $$
BEGIN
  NEW.seq := nextval('change_seq');
  RETURN NEW;
END $$ LANGUAGE plpgsql;
-- attach to every synced table:
-- CREATE TRIGGER t_seq BEFORE INSERT OR UPDATE ON <table>
--   FOR EACH ROW EXECUTE FUNCTION bump_seq();

CREATE TYPE plan_t        AS ENUM ('free','plus');
CREATE TYPE habit_mode_t  AS ENUM ('tap','check','timer','number');
CREATE TYPE habit_kind_t  AS ENUM ('build','limit');
CREATE TYPE cadence_t     AS ENUM ('daily','weekly_total');
CREATE TYPE goal_type_t   AS ENUM
('daily_target','streak','weekly_total','count','limit','savings');
CREATE TYPE goal_status_t AS ENUM
('active','due','completed','claimed','archived');
CREATE TYPE treat_kind_t  AS ENUM ('free','paid');
CREATE TYPE energy_t      AS ENUM ('low','medium','high');
CREATE TYPE journal_kind_t AS ENUM ('free','gratitude');
CREATE TYPE channel_t     AS ENUM ('push','email');
```

2. Identity and settings

Guests have no server rows; a row appears only after sign-in. The Auth.js adapter tables (accounts, sessions) are managed by the library and not listed; the tables below are TIKO's own.

```
CREATE TABLE users (
  id               uuid PRIMARY KEY,
  email            citext NOT NULL UNIQUE,
  name             text,
  time_zone        text NOT NULL DEFAULT 'Asia/Kolkata',
  currency         char(3) NOT NULL DEFAULT 'INR',
  plan             plan_t NOT NULL DEFAULT 'free',
  softened_until   date,                       -- low-mood mode (FR-MND-08)
  streaks_hidden   boolean NOT NULL DEFAULT false,  -- FR-ENG-09
  softening_on     boolean NOT NULL DEFAULT true,
  created_at       timestamptz NOT NULL DEFAULT now(),
  delete_requested_at timestamptz,             -- FR-SET-03
  purge_after      timestamptz,                -- delete_requested_at + 30
days at most
  CHECK (currency ~ '^[A-Z]{3}$')
);

-- Magic links: stored hashed, 15 minutes, single use (FR-ACC-04)
CREATE TABLE magic_links (
  token_hash  bytea PRIMARY KEY,
  email       citext NOT NULL,
  expires_at  timestamptz NOT NULL,
  used_at     timestamptz,
  created_at  timestamptz NOT NULL DEFAULT now(),
  CHECK (expires_at <= created_at + interval '15 minutes')
);

-- Per-user data key for journal encryption, wrapped by the master key
CREATE TABLE user_keys (
  user_id      uuid NOT NULL REFERENCES users ON DELETE CASCADE,
  key_version  smallint NOT NULL,
  wrapped_key  bytea NOT NULL,
  created_at   timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, key_version)
);

-- Plan limits as data, so billing in V1 only changes rows
CREATE TABLE entitlements (
  plan          plan_t PRIMARY KEY,
  max_habits    int,   -- null = unlimited
  max_goals     int,
```

```
  features      jsonb NOT NULL DEFAULT '{}'
);
-- free: 5 habits, 2 goals. A hard cap of 5 active goals applies to every
plan (FR-GOL-03).
```

Billing tables (orders, payments, webhook events for Razorpay) are added in V1 and are out of scope here.

3. Habits, goals and treats

The seven core habits are a catalog, not seeded rows: a habit row is created when the user activates one, keyed by `core_key` so a guest merge never duplicates it. Goal templates and the 30 starter treats are static data shipped with the app, not tables.

```
-- Synced tables carry the common columns from section 1 (id, user_id,
-- created_at, updated_at, deleted_at, seq); shown here only where useful.

CREATE TABLE habits (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  core_key     text,                       --
'water','gym','steps','deep_work','meditation','journal','reading'
  name         text NOT NULL,
  icon         text NOT NULL,
  mode         habit_mode_t NOT NULL,
  unit         text,
  target       numeric(12,2) NOT NULL CHECK (target > 0),
  kind         habit_kind_t NOT NULL DEFAULT 'build',
  cadence      cadence_t NOT NULL DEFAULT 'daily',
  paused_until date,                       -- FR-HAB-09
  archived_at  timestamptz,                -- history kept (FR-HAB-05)
  sort_order   int NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL,
  UNIQUE (user_id, core_key)
);

CREATE TABLE habit_logs (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  habit_id     uuid NOT NULL REFERENCES habits,
```

```
  value        numeric(12,2) NOT NULL CHECK (value >= 0),   -- glasses,
minutes, steps, 1
  local_date   date NOT NULL,
  logged_at    timestamptz NOT NULL,
  source       text NOT NULL DEFAULT 'manual',   -- 'manual' now; 'health' in
V1
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL
);

CREATE TABLE treats (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  name         text NOT NULL,
  starter_key  text,                     -- set when copied from the 30
starter treats
  photo_ref    text,                     -- object-storage key
  cost_minor   bigint CHECK (cost_minor >= 0),
  kind         treat_kind_t NOT NULL DEFAULT 'free',
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL
);

CREATE TABLE goals (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  type         goal_type_t NOT NULL,
  title        text NOT NULL,
  habit_id     uuid REFERENCES habits,           -- habit goals
  category_id  uuid,                             -- spending goals (FK to
categories)
  target       numeric(14,2) NOT NULL CHECK (target > 0),
  period_days  int,                              -- e.g. 7 days
  start_date   date NOT NULL,
  end_date     date,
  status       goal_status_t NOT NULL DEFAULT 'active',
  progress     numeric(14,2) NOT NULL DEFAULT 0, -- cache, recomputed from
logs
  nudged_50    boolean NOT NULL DEFAULT false,   -- FR-GOL-06
  nudged_90    boolean NOT NULL DEFAULT false,
  softened_factor numeric(3,2),                  -- e.g. 0.75 while softened
```

```
  treat_id     uuid REFERENCES treats ON DELETE SET NULL,  -- optional (FR-
TRT-03)
  completed_at timestamptz,
  claimed_at   timestamptz,                      -- Trophy shelf date (FR-
TRT-05)
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL,
  CHECK (habit_id IS NOT NULL OR category_id IS NOT NULL OR type = 'savings')
);

-- Savings contributions (V1, P2): manual only; TIKO never moves money
CREATE TABLE goal_contributions (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  goal_id      uuid NOT NULL REFERENCES goals ON DELETE CASCADE,
  amount_minor bigint NOT NULL CHECK (amount_minor > 0),
  local_date   date NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL
);
```

Notes. There is no failed status, so FR-GOL-08 cannot be broken by a bug. The five-active-goal cap and the plan caps are enforced in the API and the client; a deferred constraint trigger on `goals` is a second line of defense. Habit target decimals allow values like 0.5 liters while storing counts exactly.

4. Money

TIKO records spending only; there is no account, balance or bank table (FR-MON-13). Paid treats are flagged on the expense so they draw from the treat budget and stay out of ordinary spending (FR-TRT-06).

```
CREATE TABLE categories (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  name           text NOT NULL,
  default_key    text,                -- food, transport, bills, shopping,
fun, health, education, other
  monthly_limit_minor bigint CHECK (monthly_limit_minor > 0),  -- FR-MON-05
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
```

```
NULL,
  deleted_at timestamptz, seq bigint NOT NULL
);
CREATE UNIQUE INDEX categories_default_uq ON categories (user_id,
default_key) WHERE default_key IS NOT NULL;
CREATE UNIQUE INDEX categories_name_uq ON categories (user_id, lower(name))
WHERE deleted_at IS NULL;
ALTER TABLE goals ADD FOREIGN KEY (category_id) REFERENCES categories;

CREATE TABLE expenses (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  amount_minor   bigint NOT NULL CHECK (amount_minor > 0),   -- paise (FR-
MON-03)
  category_id    uuid NOT NULL REFERENCES categories,
  local_date     date NOT NULL,                              -- defaults to
today in the app
  note           text,
  is_income      boolean NOT NULL DEFAULT false,             -- FR-MON-10
(P2)
  is_treat       boolean NOT NULL DEFAULT false,             -- paid treat,
excluded from spending
  treat_id       uuid REFERENCES treats ON DELETE SET NULL,
  recurring_id   uuid,                                       -- FK added
below
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL
);

CREATE TABLE budgets (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  month               date NOT NULL CHECK (month = date_trunc('month',
month)::date),
  amount_minor        bigint NOT NULL CHECK (amount_minor >= 0),
  treat_budget_minor  bigint NOT NULL DEFAULT 0 CHECK (treat_budget_minor >=
0),
  warned_categories   uuid[] NOT NULL DEFAULT '{}',          -- 80% warning
shown once per month
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL,
```

```
  UNIQUE (user_id, month)
);

CREATE TABLE recurring_expenses (            -- P1; moves to V1 if time runs
short
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  amount_minor   bigint NOT NULL CHECK (amount_minor > 0),
  category_id    uuid NOT NULL REFERENCES categories,
  day_of_month   smallint NOT NULL CHECK (day_of_month BETWEEN 1 AND 31),
  last_posted    date,
  active         boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL
);
ALTER TABLE expenses ADD FOREIGN KEY (recurring_id) REFERENCES
recurring_expenses ON DELETE SET NULL;
```

Safe-to-spend is computed, never stored: spent this month is the sum of `amount_minor` where `is_income` and `is_treat` are both false, from the first day of the month in the user's time zone. Where day 29 to 31 does not exist in a month, a recurring item posts on the last day. The treat budget has no UI spec yet (open question 10 in the Technical Specification), so `treat_budget_minor` is the proposed home.

5. Mind, engagement, notifications and sync

```
-- MIND ------------------------------------------------------------
CREATE TABLE mood_entries (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  score        smallint NOT NULL CHECK (score BETWEEN 1 AND 5),   -- 5-point
face scale
  energy       energy_t NOT NULL,
  tags         text[] NOT NULL DEFAULT '{}' CHECK (cardinality(tags) <= 3),
  note         text CHECK (note IS NULL OR (length(note) <= 140 AND note !~
E'\n')),
  local_date   date NOT NULL,
  slot         smallint NOT NULL DEFAULT 0,        -- one entry per check-in
time (FR-MND-03)
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
```

```
NULL,
  deleted_at timestamptz, seq bigint NOT NULL,
  UNIQUE (user_id, local_date, slot)
);

CREATE TABLE journal_entries (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  kind             journal_kind_t NOT NULL DEFAULT 'free',
  body_ciphertext  bytea NOT NULL,                -- AES-256-GCM; never
plaintext
  nonce            bytea NOT NULL,
  key_version      smallint NOT NULL,
  local_date       date NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL,
  FOREIGN KEY (user_id, key_version) REFERENCES user_keys
);

-- ENGAGEMENT ------------------------------------------------------
CREATE TABLE xp_events (                          -- append-only; XP is
never removed
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  source_type  text NOT NULL,                     --
'habit_log','mood','journal','goal_claim'
  source_id    uuid NOT NULL,
  points       int NOT NULL CHECK (points > 0),
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT
NULL,
  deleted_at timestamptz, seq bigint NOT NULL,
  UNIQUE (user_id, source_type, source_id)          -- an edit or retry
cannot award twice
);
-- Level and total XP are computed: SUM(points) mapped through a level table
in app config.

CREATE TABLE badges_earned (                      -- P2, about 25 badges at
V1
  user_id uuid NOT NULL REFERENCES users ON DELETE CASCADE,
  badge_key text NOT NULL, earned_at timestamptz NOT NULL,
  PRIMARY KEY (user_id, badge_key)
```

```
);

-- NOTIFICATIONS ---------------------------------------------------
CREATE TABLE notification_prefs (
  user_id        uuid PRIMARY KEY REFERENCES users ON DELETE CASCADE,
  reminder_time  time,
  tone           text NOT NULL DEFAULT 'gentle' CHECK (tone IN
('gentle','playful','minimal')),
  quiet_start    time NOT NULL DEFAULT '22:00',
  quiet_end      time NOT NULL DEFAULT '08:00',
  push_on        boolean NOT NULL DEFAULT true,
  email_on       boolean NOT NULL DEFAULT true,
  updated_at     timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE push_subscriptions (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  endpoint text NOT NULL UNIQUE, p256dh text NOT NULL, auth text NOT NULL,
  last_ok_at timestamptz, created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE notification_log (                    -- enforces 2 a day and
the 7-day gap
  id bigserial PRIMARY KEY,
  user_id uuid NOT NULL REFERENCES users ON DELETE CASCADE,
  kind text NOT NULL,                              --
'reminder','reengage','recap'
  channel channel_t NOT NULL,
  sent_at timestamptz NOT NULL DEFAULT now()
);

-- SYNC ------------------------------------------------------------
CREATE TABLE devices (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users ON DELETE
CASCADE,
  last_seq bigint NOT NULL DEFAULT 0, last_seen_at timestamptz NOT NULL
DEFAULT now()
);

CREATE TABLE applied_ops (                         -- idempotency; purged
after 30 days
```

```
  op_id uuid PRIMARY KEY,
  user_id uuid NOT NULL REFERENCES users ON DELETE CASCADE,
  applied_at timestamptz NOT NULL DEFAULT now()
);
```

Notes. The journal's `FOREIGN KEY` to `user_keys` guarantees every ciphertext has a key row to decrypt it. Journal search runs on the device over the decrypted local copy, so there is no text index on the server. The low-mood state is derived from `mood_entries` and stored only as `users.softened_until`.

6. Relationships, constraints and indexes

Everything hangs off `users`. Habits and categories are the two parents that goals, logs and expenses point to.

Parent	Child	Cardinality	On delete
users	every user-owned table	1 to many	Cascade (account deletion)
habits	habit_logs	1 to many	Restrict; habits are archived, never hard-deleted, so history survives
habits	goals	1 to many	Restrict
categories	expenses, recurring_expenses, goals	1 to many	Restrict; delete is a tombstone
treats	goals, expenses	1 to many	Set null; removing a treat never removes a goal
goals	goal_contributions	1 to many	Cascade
recurring_expenses	expenses	1 to many	Set null
users, key_version	journal_entries	many to 1	Restrict until re-encryption

Integrity rules enforced in the database

- Positive money and targets (`CHECK > 0`), mood score 1 to 5, at most 3 mood tags, a one-line note, one mood entry per (user, date, slot), one budget per month, one row per core habit per user.
- No failed goal state exists in the enum, so the "never fail" rule cannot be violated by a bug.
- Journal rows cannot exist without a ciphertext, nonce and key row.

Rules enforced in the application (and tested there)

- At most 5 active goals, plus the free-plan caps of 5 habits and 2 goals (FR-GOL-03).
- Edits and deletes of logs only within 7 days (FR-HAB-10); the 10-second undo.
- Streak, goal progress, safe-to-spend and level, all computed in the shared engine.

```
-- Change feed: every synced table
CREATE INDEX habits_seq        ON habits (user_id, seq);
CREATE INDEX habit_logs_seq    ON habit_logs (user_id, seq);
CREATE INDEX goals_seq         ON goals (user_id, seq);
CREATE INDEX treats_seq        ON treats (user_id, seq);
CREATE INDEX expenses_seq      ON expenses (user_id, seq);
CREATE INDEX categories_seq    ON categories (user_id, seq);
CREATE INDEX budgets_seq       ON budgets (user_id, seq);
CREATE INDEX mood_entries_seq  ON mood_entries (user_id, seq);
CREATE INDEX journal_seq       ON journal_entries (user_id, seq);
CREATE INDEX xp_events_seq     ON xp_events (user_id, seq);

-- Reads by day and by month
CREATE INDEX habit_logs_day    ON habit_logs (user_id, habit_id, local_date)
WHERE deleted_at IS NULL;
CREATE INDEX expenses_day      ON expenses (user_id, local_date) WHERE
deleted_at IS NULL;
CREATE INDEX mood_day          ON mood_entries (user_id, local_date) WHERE
deleted_at IS NULL;

-- Goal engine: the handful of live goals per user
CREATE INDEX goals_live        ON goals (user_id, habit_id) WHERE status IN
('active','due','completed');

-- Jobs
```

```
CREATE INDEX notif_log_day     ON notification_log (user_id, sent_at DESC);
CREATE INDEX applied_ops_age   ON applied_ops (applied_at);
CREATE INDEX users_purge       ON users (purge_after) WHERE purge_after IS
NOT NULL;
```

7. Sync feed, retention, deletion and access control

Push and pull. A push applies each mutation once: look up `op_id` in `applied_ops`, skip if present, otherwise upsert the row when the incoming `updated_at` is newer than the stored one, then record the op, all in one transaction. A pull reads each synced table with `seq > cursor` for the user and returns the highest `seq` seen.

```
-- apply one mutation (simplified)
INSERT INTO habit_logs (id, user_id, habit_id, value, local_date, logged_at,
updated_at, deleted_at)
VALUES ($1,$2,$3,$4,$5,$6,LEAST($7, now() + interval '5 minutes'),$8)
ON CONFLICT (id) DO UPDATE
  SET value = EXCLUDED.value, deleted_at = EXCLUDED.deleted_at, updated_at =
EXCLUDED.updated_at
  WHERE habit_logs.user_id = EXCLUDED.user_id
    AND habit_logs.updated_at < EXCLUDED.updated_at;   -- latest edit wins

-- pull
SELECT * FROM habit_logs WHERE user_id = $1 AND seq > $2 ORDER BY seq LIMIT
500;
```

Known edge. A sequence value is taken before a transaction commits, so a lower `seq` can become visible after a higher one and be skipped. The client therefore re-pulls from `cursor - 100` and ignores rows it already holds (same id and `seq`). Transactions here are tiny, so the window is small.

Retention and deletion

Data	Kept for	How
Account after deletion request	Access ends at once; rows purged within 30 days, targeted within 7	`users.purge_after`, nightly job runs `DELETE FROM users`, cascade removes the rest
Database backups	30 days at most	Provider retention setting; needed to meet NFR-07

Data	Kept for	How
Tombstones (`deleted_at`)	90 days, then physically removed	Nightly job; a device unseen for 90 days gets `cursor_too_old` and does a full resync
applied_ops	30 days	Nightly job
notification_log	90 days	Nightly job
magic_links	1 day after expiry or use	Nightly job
Exports	Not stored	Streamed on request, journal decrypted for the owner only

Access control

The app connects as a role with no DDL rights and scopes every query by `user_id`. Row-level security is a second lock and works on Neon or Supabase by setting the user for each transaction:

```
ALTER TABLE habit_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY own_rows ON habit_logs
  USING (user_id = current_setting('app.user_id')::uuid)
  WITH CHECK (user_id = current_setting('app.user_id')::uuid);
-- per request: SET LOCAL app.user_id = '<session user>';
-- repeat for every user-owned table
```

- A separate migration role owns the schema; the app role cannot alter it.
- Any analytics or reporting role is denied `journal_entries`, `mood_entries` and `expenses`, matching the rule that mind data and exact amounts never reach analytics (NFR-06).
- The master key that wraps `user_keys` is never stored in the database.

8. Sizing, migrations and seed data

At the NFR-12 target of 50,000 users, one Postgres instance holds roughly 10 GB of new data a year, so a single database is enough with the indexes above. Figures are approximate and rest on an assumption to validate with beta data: 30% of users active daily (15,000), writing about 12 rows each per day.

Table	Rows per active user per day	Rows per year	Bytes per row with indexes	Size per year
habit_logs	6	33 million	150	4.9 GB
xp_events	3	16 million	120	2.0 GB
expenses	2	11 million	170	1.9 GB
mood_entries	0.7	3.8 million	150	0.6 GB
journal_entries	0.3	1.6 million	700	1.1 GB
Total	12	66 million		10.5 GB

Free-tier storage will be far smaller than this, so plan a paid database before public launch. If habit_logs or expenses pass about 100 million rows, partition them by year on `local_date`; nothing in the schema blocks that. XP could be aggregated per habit per day if xp_events grows faster than expected.

Migrations

- Drizzle Kit generates forward-only SQL migrations that live in the repository; an applied migration is never edited.
- Use expand then contract: add columns as nullable, deploy code that writes both, backfill, then drop the old column in a later release. This matters because older cached clients keep running after a deploy.
- Each sync request carries `schema_v`. A client below the minimum supported version gets a `client_too_old` response and is asked to reload, so a stale service worker cannot write rows the server no longer understands.
- The client database is versioned the same way with Dexie upgrade steps, tested against fixtures from each prior version.
- CI applies all migrations to an empty Postgres and runs the constraint, sync-idempotency and account-deletion cascade tests against it.

Seed and static data

Data	Where it lives	Notes
Core habits (7)	App code	Catalog; a row is created on activation, keyed by `core_key`

Data	Where it lives	Notes
Goal templates (at least 10)	App code	Each carries a goal type, target and period
Starter treats (30)	App code	Sorted by cost; copied into `treats` with `starter_key` when chosen
Default categories (8)	Created per user on first sync	Client-generated ids, unique per user on `default_key`
Level names and XP thresholds	App config	Names and thresholds are not yet defined in the documents
Entitlements	Database seed	Two rows: free and plus